8. Linear Regression

Best-Fitting Line for the Best Prediction

Previous Section:

7. Supervised & Unsupervised Learning

Contents:

This section is about Linear Regression—one of the most popular models in Machine Learning. It's used when we want to **predict numerical values**, and it does that using a surprisingly simple equation, we'll get to that in a moment.

But before we dive into the model itself, there's one idea you need to keep in mind:

> **Regression means predicting a number.**

That's it.

If the output is something like a price, a score, a temperature, or any continuous value—you're dealing with a regression problem.

Now, Linear Regression is the simplest way to approach this. It tries to find a relationship between variables, and more specifically, it tries to draw a line that best represents that relationship.

Not just any line—but the one that fits the data **as closely as possible**.

So the real question becomes:

> **How do we find the best-fitting line for making predictions?**

That's exactly what we're about to explore.

1. Linear Regression - The Best-Fitting Line

Among all the possible lines we could draw, there is always one that gets as close as possible to the data. That line is called the **Best-Fitting Line,** or more formally, the **Linear Regression line**. And the entire idea of Linear Regression can be summarized with a very simple equation:

$$Y = A + BX$$

We'll stick to this form throughout the course—so whenever you see Linear Regression, this is the equation we're talking about.

Let's break it down its component:

- Y **is t**he predicted value: This is what the model is trying to estimate (also called the *target* or *dependent variable*)
- X is the input value: The information we already have (also called the *feature*, *independent variable*, or *predictor*)
- A **is called t**he intercept. This is the value of Y **when** $X = 0$**:** You can think of it as the **starting point** or baseline prediction
- B **is called t**he slope: This tells us **how much** Y **changes when** X **changes.** It captures both the **strength** and the **direction** of the relationship

So in simple terms:

- A **sets where the line starts**
- B **controls how the line moves**

And once we know these two values, we can draw the line—and start making predictions.

2. Linear Regression - The Formulas

Now, to actually find A and B, we use the following formulas:

$$B = \frac{n\sum x_i y_i - \sum x_i \sum y_i}{n\sum x_i^2 - (\sum x_i)^2} \quad , \quad A = \frac{\sum y_i - B\sum x_i}{n}$$

At first glance, this might look a bit intimidating. But don't worry—we're going to walk through a concrete example right after this, and you'll see that it's actually very mechanical and straightforward.

- Just a quick note: n is the number of samples (how many data points you have)

And you might notice there are multiple ways to write these formulas. That's true—but throughout this course, we'll stick with this **final form**, because it's the most practical when working with real data.

So for now, don't try to memorize everything. Just keep one idea in mind:

> **These formulas are simply a way to calculate the best-fitting line from your data.**

Consider this following data:

Height (cm)	Weight (kg)
150	45
155	50
158	52
160	54
162	56
165	58
168	61
170	63
172	65
175	68
178	70

Height (cm)	Weight (kg)
180	72
182	74
185	78
188	80

This is a **2D dataset**, because we are working with two variables:

- **Height** → X **(input)**
- **Weight** → Y **(target)**

The very first thing you should always do is:

- **Identify the target variable** → In this case, **Weight (Y)** is what we want to predict.

Optionally—but highly recommended—you can visualize the data:

- Plot a **scatter plot** using tools like Python (Pandas / Matplotlib / Seaborn), MATLAB, R, or even Excel
- Place **Height on the X-axis** and **Weight on the Y-axis**

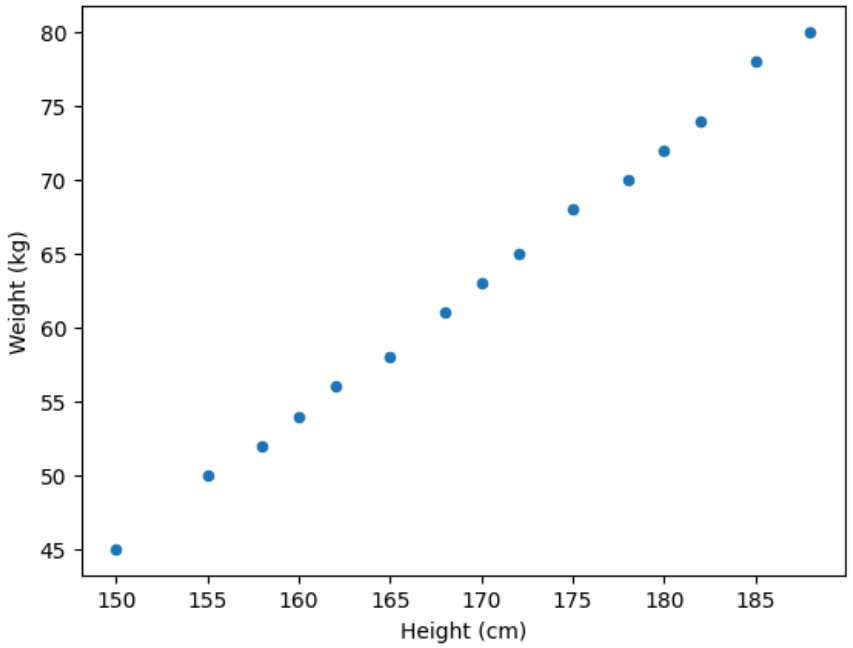
Once you do that, you'll get a cloud of points. Here, I used a Python code, you can simply copy it to whatever Python IDE you are using:

```
import pandas as pd

data = {
    "Height (X)": [150, 155, 158, 160, 162, 165, 168, 170,
172, 175, 178, 180, 182, 185, 188],
    "Weight (Y)": [45, 50, 52, 54, 56, 58, 61, 63, 65, 68,
70, 72, 74, 78, 80]
}

# Plot
df = pd.DataFrame(data)
# Linear Regression line
df.plot(kind='scatter', x='Height (X)', y='Weight (Y)')
```

Output:



→ This type of visualization is called a **Scatter Plot**

Now, this is where Linear Regression becomes interesting.

Looking at the scatter plot, you'll notice something:

- The points seem to follow a **trend**
- You could draw a line through them... actually, you could draw **many** lines

And every line follows the same form $Y = A + BX$ that we are introduced earlier

But here's the key question: **Which line is the best one?**

Well, for any line you draw:

- You can plug in X **(Height)** into the equation
- You get a **predicted** Y **(Weight)**

Now compare that to the **actual** Y from the dataset. The difference between them is called the **error.** So to measure how good a line is, we:

1. Take the difference between predicted and actual values
2. Square those differences (to avoid negatives)

3. Add them all together

This gives us → The **Sum of Squared Errors (SSE)**

We can formalize it like this:

$$D^2 = \sum_{i=1}^{n} (y_i - y_i')^2$$

Now let's interpret it in a very simple way:

- y_i → the actual value (from the dataset)
- y_i' → the predicted value (from our line)
- $(y_i - y_i')$ → the error for each point

So:

- $(y_i - y_i')^2$ is just the **squared error of one data point**
- And when we sum over all points → we get the **total error (SSE)**

Now you might wonder: **Why do we square it?**

Two main reasons:

- First, to **avoid negativity.** Errors can be positive or negative, and we don't want them to cancel each other out
- Second, it's just **better behaved mathematically.** Squaring makes optimization smoother and easier to work with (***and it is a bit cooler than absolute value!!***)

We're not just measuring error—we're measuring it in a way that makes it easier to **find the best-fitting line**.

So now the problem becomes very clear. Every line gives a different SSE. Some lines are worse (large error). Some lines are better (small error)

→ **The Best-Fitting Line is the one where SSE is as small as possible.**

That's the whole idea of Linear Regression. It's not about drawing a random line—it's about finding the values of A and B such that the prediction error is minimized. This is what we call **optimization.**

3. Linear Regression - Handful Calculations

Now, let's actually compute the two things we care about:

- **The slope (B)**
- **The intercept (A)**

Since we're working with data in Python, we'll follow a **Pandas-style workflow** to extract everything we need. These steps are not strictly fixed—but this is a **very practical standard approach** (you might want to note this down).

Step-by-step plan

- **Step 0:** Determine the number of samples → Using `len()`

```
n = len(df)
print("Number of samples:", n)
```

Number of samples: 15

→ $n = 15$

- **Step 1:** Compute XY **(element-wise)** → Then take the **sum of all** XY

```
df['XY'] = df['Height (X)'] * df['Weight (Y)']
print(df)
sum_xy = sum(df['XY'])
print("\nSum of XY:", sum_xy)
```

	Height (cm)	Weight (kg)	XY
0	150	45	6750
1	155	50	7750
2	158	52	8216
3	160	54	8640
4	162	56	9072
5	165	58	9570
6	168	61	10248
7	170	63	10710
8	172	65	11180
9	175	68	11900
10	178	70	12460
11	180	72	12960
12	182	74	13468
13	185	78	14430
14	188	80	15040

Sum of XY: 162394

→ $\sum XY = 162394$

- **Step 2a:** Compute **Sum of X and Mean of X**

```
sum_x = sum(df['Height (X)'])
mean_x = sum_x / n
print("Sum of X:", sum_x)
print("Mean of X:", mean_x)
```

Sum of X: 2548
Mean of X: 169.86666666666667

→ $\sum X = 2548; \bar{X} = 169.87$

- **Step 2b:** Compute **Sum of Y and Mean of Y**

```
sum_y = sum(df['Weight (Y)'])
mean_y = sum_y / n
print("Sum of Y:", sum_y)
print("Mean of Y:", mean_y)
```

Sum of Y: 946
Mean of Y: 63.06666666666667

$\rightarrow \sum Y = 946; \bar{Y} = 63.07$

- **Step 3a:** Compute **Sum of X^2** (each X squared, then summed)

```
squared_sum_x = sum(df['Height (X)']**2)
print("Squared sum of X:", squared_sum_x)
```

Squared sum of X: 434668

$\rightarrow \sum X^2 = 434668$

- **Step 3b:** Compute **Sum of Y^2** (each Y squared, then summed)

```
squared_sum_y = sum(df['Weight (Y)']**2)
print("Squared sum of Y:", squared_sum_y)
```

Squared sum of Y: 61228

$\rightarrow \sum Y^2 = 61228$

- **Step 4:** Compute **(Sum of X)^2**

```
sum_squared_x = sum_x**2
print("Sum of squared X:", sum_squared_x)
```

Sum of squared X: 6492304

→ $(\sum X)^2 = 6492304$

Important: **Step 3a (ΣX^2)** and **Step 4 ($(\Sigma X)^2$)** are **NOT the same thing**

So how many values do we have? If I count, that's around **8–9 key values.**

And once we have all of these, we can plug them directly into the formulas for A and B, and finally get our **best-fitting line**.

We'll translate all of this into actual code—and you'll see how clean this becomes in Pandas:

```
# Find the best fitting line
B = (n * sum_xy - sum_x * sum_y) / (n * squared_sum_x - sum
_squared_x)
A = (sum_y - B * sum_x) / n
print(f"The intercept A: {A}")
print(f"The slope B: {B}")
print(f"The best-fitting line: Y = {A} + {B}X")
```

The intercept A: -93.23076923076924
The slope B: 0.9201183431952663
The best-fitting line: Y = -93.23076923076924 + 0.9201183431952663X

→ Our best-fitting line is (rounded up by two decimals): $Y = -93.23 + 0.92X$

See? Much easier than staring at the formula, right?

Instead of worrying about all those summations, we just let the code handle it—and we directly get the two values that define our line: A and B.

We finally have what we need → A complete **best-fitting line**

Now comes the fun part: Making predictions

We take this equation and **plug in X (Height)**:

- For each value of X
- We compute a **Predicted Y**

→ This is what the model *thinks* the weight should be

Then we compare:

- **Actual Y (real weight)**
- **Predicted Y (from the line)**

And that difference is exactly what we used earlier to compute the **error (SSE)**

```
df['Predicted_Weight (Y_pred)'] = A + B * df['Height (X)']
df['Actual_Weight (Y)'] = df['Weight (Y)']
print(df[['Height (X)', 'Predicted_Weight (Y_pred)', 'Actua
l_Weight (Y)']])
```

	Height (X)	Predicted_Weight (Y_pred)	Actual_Weight (Y)
0	150	44.786982	45
1	155	49.387574	50
2	158	52.147929	52
3	160	53.988166	54
4	162	55.828402	56
5	165	58.588757	58
6	168	61.349112	61
7	170	63.189349	63
8	172	65.029586	65
9	175	67.789941	68
10	178	70.550296	70
11	180	72.390533	72
12	182	74.230769	74
13	185	76.991124	78
14	188	79.751479	80

Once we have the p**redicted values (from the model) and the actual values (from the dataset)**, we can measure how good our model is with two common

parameters:

- **SSE** → total error
- **MSE** → average error

```
squared_errors = (df['Predicted_Weight (Y_pred)'] - df['Act
ual_Weight (Y)']) ** 2
print("Sum of Squared Error (SSE):", sum(squared_errors))
print("Mean Squared Error (MSE):", sum(squared_errors) / n)
```

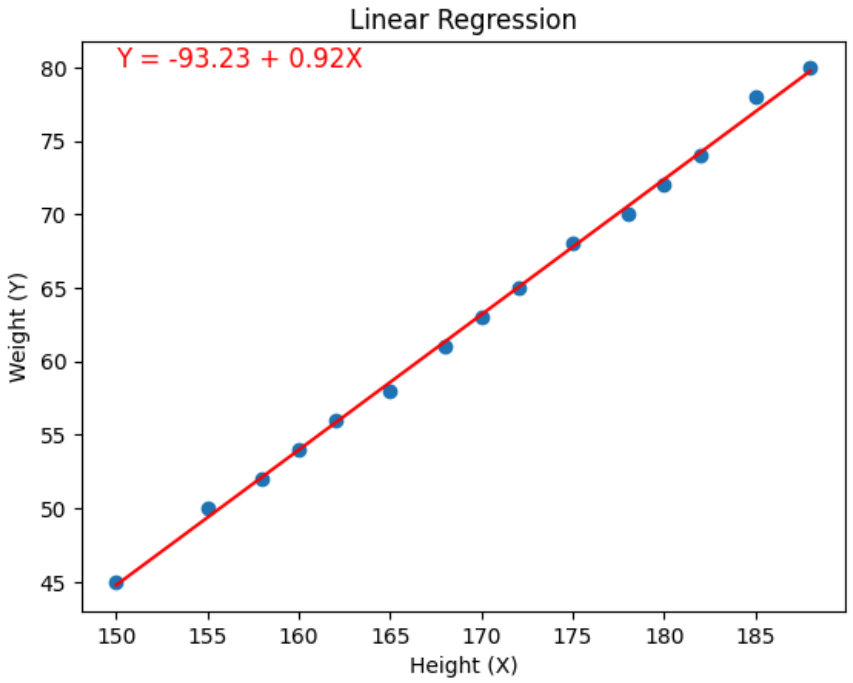
```
Sum of Squared Error (SSE): 2.6094674556213238
Mean Squared Error (MSE): 0.17396449704142158
```

The smaller these values are, the better our line fits the data.

Visualizing the result (because why not??)

```
import matplotlib.pyplot as plt

# Draw the regression line
plt.scatter(df['Height (X)'], df['Weight (Y)'], label='Actu
al Data')
plt.plot(df['Height (X)'], df['Predicted_Weight (Y_pred)'],
color='red', label='Regression Line')
plt.text(x=df['Height (X)'].min(), y=df['Weight (Y)'].max
(),
         s=f'Y = {A:.2f} + {B:.2f}X', fontsize=12, color='r
ed')
plt.xlabel('Height (X)')
plt.ylabel('Weight (Y)')
plt.title('Linear Regression')
```



Now you'll clearly see:

- The **blue dots** → actual data
- The **red line** → our best-fitting line

Because of how simple—but powerful—it is, Linear Regression is used everywhere: Statistics; Machine Learning; Data Science; Data Analytics

And the main purpose? **Predicting future values based on patterns in data**

4. Multiple Linear Regression

So far, we only used **one feature (X)**. But in reality, datasets rarely look like that.

- You usually have **multiple features**
- Multiple factors influencing the outcome

Good news is: Linear Regression can still handle it.

But instead of one slope, we now have **many slopes**:

$$Y = A + B_1X_1 + B_2X_2 + B_3X_3 + \cdots + B_nX_n$$

Each feature gets its own coefficient.

And here's where things change: We can no longer use the simple formula from before.

Instead, we switch to a **matrix-based solution**:

$$B = (X^T X)^{-1} X^T Y$$

- X^T → Transpose of X
- $(X^T X)^{-1}$ → Matrix Inverse
- This involves **dot products and linear algebra**

Expanding our dataset (because it is expandable)

Height (cm)	Weight (kg)	Resting Heart Rate (BPM)	Daily Calories (Est.)
150	45	68	1393
155	50	81	1654
158	52	76	1617
160	54	72	1681
162	56	69	1816
165	58	82	1664
168	61	68	1779
170	63	80	1959
172	65	84	1978
175	68	72	2207
178	70	72	2149
180	72	82	2162
182	74	65	2514
185	78	69	2473
188	80	64	2566

Now we're stepping into a more realistic scenario.

This time, our dataset has **multiple features**:

- **Height** → X_1
- **Weight** → X_2
- **Resting Heart Rate** → X_3

And we change the target: **Daily Calories** → Y

So instead of a single input, we now have **three inputs** affecting one output.

That means our equation becomes: $Y = A + B_1X_1 + B_2X_2 + B_3X_3$

For simplicity:

- Treat all features as X_1, X_2, X_3 as we defined above
- And the target as Y

How do we solve this? We can't use the previous "hand-calculation" method anymore.

Instead, we use the **matrix formula**: $B = (X^TX)^{-1}X^TY$

This gives us all coefficients at once: A, B_1, B_2, B_3

- **Step 1: Prepare the data**

Since this method requires matrix operations → We convert our data into **NumPy arrays**

```
import pandas as pd

data = {
    'Height (cm)': [150, 155, 158, 160, 162, 165, 168, 170,
172, 175, 178, 180, 182, 185, 188],
    'Weight (kg)': [45, 50, 52, 54, 56, 58, 61, 63, 65, 68,
70, 72, 74, 78, 80],
    'Resting Heart Rate (BPM)': [68, 81, 76, 72, 69, 82, 6
8, 80, 84, 72, 72, 82, 65, 69, 64],
    'Daily Calories (Est.)': [1393, 1654, 1617, 1681, 1816,
1664, 1779, 1959, 1978, 2207, 2149, 2162, 2514, 2473, 2566]
}
```

```
df = pd.DataFrame(data)
# Convert to numpy
X = df[['Height (cm)', 'Weight (kg)', 'Resting Heart Rate
(BPM)']].to_numpy()
Y = df['Daily Calories (Est.)'].to_numpy()
```

The formula assumes a bias term (intercept A). So we need to add a column of **1s** to X:

```
import numpy as np

# Add a bias term -> To learn the intercept
ones = np.ones((X.shape[0], 1))
X = np.hstack((ones, X))
```

Now X looks like: $[1, X_1, X_2, X_3]$

→ This "1" is what allows us to learn A

- **Step 2: Apply the formula**

```
B = np.linalg.inv(X.T @ X) @ X.T @ Y
```

```
[ 8.50687551e+03 -8.11114080e+01  1.20310579e+02 -4.64952284e+00]
```

→ These values represent our slopes: $[A, B_1, B_2, B_3]$

- **Step 3: Interpret the result**

```
# Interpret the result
A = B[0]
B1, B2, B3 = B[1], B[2], B[3]
# The best fitting equation
print(f"Best Equation: ({A}) + ({B1})X1 + ({B2})X2 + ({B3})
X3")
```

Best Equation: (8506.87551462982) + (-81.11140796714005)X1 + (120.31057928013581)X2 + (-4.649522842497902)X3

Instead of:

- Computing many sums manually
- Handling each variable separately

We let **linear algebra do everything in one step**

- **Step 4: Evaluation (How well our model perform this time around?)**

```
# Make prediction
df['Daily Calories (Est.) Actual'] = df['Daily Calories (Est.)']
df['Daily Calories (Est.) Predicted'] = A + B1 * df['Height (cm)'] + B2 * df['Weight (kg)'] + B3 * df['Resting Heart Rate (BPM)']
print(df[['Daily Calories (Est.) Actual', 'Daily Calories (Est.) Predicted']])
print("\nMean Squared Error:", np.mean((df['Daily Calories (Est.) Actual'] - df['Daily Calories (Est.) Predicted'])**2))
```

	Daily Calories (Est.) Actual	Daily Calories (Est.) Predicted
0	1393	1437.972834
1	1654	1573.524893
2	1617	1594.059442
3	1681	1691.055876
4	1816	1783.402788
5	1664	1720.245925
6	1779	1902.936759
7	1959	1925.540827
8	1978	1985.341079
9	2207	2158.732867
10	2149	2156.019801
11	2162	2187.922916
12	2514	2345.363147
13	2473	2564.673148
14	2566	2585.207697

Mean Squared Error: 4676.558670936564

This step involves:

- Making prediction on the data
- Evaluate using MSE

The MSE is quite high. But does that mean the model is bad? Not necessarily.

Because error values like MSE always depend on the **scale of the data**. In this case, we are predicting **calories**, which are already in the range of thousands—so an error in the thousands is not as shocking as it looks at first glance.

A better way to think about it is:

- How large is the error compared to the actual values?
- Does the model still capture the overall trend?

And most importantly: Linear Regression is a **simple model**. It assumes a **linear relationship**, but real-world data is often more complex than that.

So a higher error doesn't always mean failure—it might just mean the relationship isn't perfectly linear, or we need better features.

5. Linear Regression with Python

Here's how you would build a Linear Regression in Python using the Scikit-Learn library. Here I created two models:

- One for predicting the target *'Daily Calories (Est.)' using only the 'Height (cm)'*
- Another for predicting the target *'Daily Calories (Est.)' using all three features: 'Height (cm)', 'Weight (kg)' and 'Resting Heart Rate (BPM)'*

If you find this code unfit for you. Feel free to write one your own (your choice, not mine)

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Load the dataset
data = {
    'Height (cm)': [150, 155, 158, 160, 162, 165, 168, 170,
172, 175, 178, 180, 182, 185, 188],
    'Weight (kg)': [45, 50, 52, 54, 56, 58, 61, 63, 65, 68,
70, 72, 74, 78, 80],
    'Resting Heart Rate (BPM)': [68, 81, 76, 72, 69, 82, 6
8, 80, 84, 72, 72, 82, 65, 69, 64],
    'Daily Calories (Est.)': [1393, 1654, 1617, 1681, 1816,
1664, 1779, 1959, 1978, 2207, 2149, 2162, 2514, 2473, 2566]
}


df = pd.DataFrame(data)

# Split features and target
target = 'Daily Calories (Est.)'
X1 = df['Height (cm)'] # Use only Height (cm)
X2 = df.drop(target, axis=1) # Use all learning features
y = df[target]

# Train-test split for model 1
X1_train, X1_test, y1_train, y1_test = train_test_split(X1,
y, test_size=0.2,
                                                        random_
state=42)
# Normalize
scaler = StandardScaler()
X1_train = scaler.fit_transform(X1_train.values.reshape(-1,
1))
X1_test = scaler.transform(X1_test.values.reshape(-1, 1))
```

```
model1 = LinearRegression()
model1.fit(X1_train.reshape(-1, 1), y1_train)
print("Model 1 Coefficients:", model1.coef_)
print("Model 1 Intercept:", model1.intercept_)
pred_y = model1.predict(X1_test.reshape(-1, 1))
print("MSE:", mean_squared_error(y1_test, pred_y))
print("R2:", r2_score(y1_test, pred_y))

# Train-test split for model 2
X2_train, X2_test, y2_train, y2_test = train_test_split(X2,
y, test_size=0.2,
                                                        random_
state=42)
# Normalize
scaler = StandardScaler()
X2_train = scaler.fit_transform(X2_train)
X2_test = scaler.transform(X2_test)

model2 = LinearRegression()
model2.fit(X2_train, y2_train)
print("\nModel 2 Coefficients:", model2.coef_)
print("Model 2 Intercept:", model2.intercept_)
pred_y = model2.predict(X2_test)
print("MSE:", mean_squared_error(y2_test, pred_y))
print("R2:", r2_score(y2_test, pred_y))
```

```
Model 1 Coefficients: [327.12845091]
Model 1 Intercept: 1987.5
MSE: 7725.485591999677
R2: 0.9446414277211003

Model 2 Coefficients: [-828.03485652 1136.91931948  -35.58389341]
Model 2 Intercept: 1987.4999999999998
MSE: 2471.164271290817
R2: 0.9822923589337925
```

Summary

I would like to summarize Linear Regression within two paragraphs like this:

Linear Regression is one of the simplest yet most powerful ideas in Machine Learning. It works by finding a relationship between input variables and a numerical target, forming a line (or a plane in higher dimensions) that best represents the data. By minimizing the error between predicted and actual values, it allows us to make meaningful predictions from patterns.

Even though it may not always achieve perfect accuracy—especially with complex data—its strength lies in its **simplicity, interpretability, and efficiency**. It serves as the foundation for many advanced models, and understanding it gives you a strong intuition for how machines learn from data.

References

https://www.geeksforgeeks.org/machine-learning/ml-linear-regression/

https://www.geeksforgeeks.org/machine-learning/ml-multiple-linear-regression-using-python/

https://developers.google.com/machine-learning/crash-course/linear-regression

https://www.ibm.com/think/topics/linear-regression

https://www.datacamp.com/tutorial/simple-linear-regression

https://www.youtube.com/watch?v=7ArmBVF2dCs

https://www.youtube.com/watch?v=EkAQAi3a4js

http://youtube.com/watch?v=gPfgB4ew3RY&vl=en

https://www.youtube.com/watch?v=i3IadpjctWg

Next Section:

9. Decision Trees